

# Tutorial Integrasi Firebase Cloud Messaging (FCM) untuk Frontend

Dokumen ini berisi panduan untuk tim Frontend (Web & Mobile) tentang cara mengintegrasikan fitur Push Notification menggunakan FCM dengan sistem Backend CariTalent.

## 1. Skenario Alur Kerja (Bagaimana Notifikasi Bekerja)

1. **User Login:** User (Talent atau EO) masuk ke dalam aplikasi.
2. **Request Token:** Setelah login berhasil, aplikasi Frontend meminta izin (permission) untuk menampilkan notifikasi kepada user.
3. **Mendapatkan Token FCM:** Jika diizinkan, Frontend akan meminta **FCM Device Token** langsung dari server Google Firebase.
4. **Update ke Backend:** Frontend mengirimkan FCM Token yang didapat tersebut ke Backend CariTalent melalui endpoint `PUT /api/v1/users/fcm-token`.
5. **Backend Bekerja:** Ketika ada kejadian penting (misal: Talent mendapat tawaran lamaran), Backend CariTalent akan menyimpan notifikasi di database dan secara otomatis menyuruh Firebase mengirim pesan push ke FCM Token tersebut.
6. **User Menerima Push:** HP/Browser user akan memunculkan pop-up atau banner notifikasi tanpa harus membuka halaman notifikasi secara manual.

## 2. PERSIAPAN WAJIB DI FIREBASE CONSOLE (Untuk Tim FE)

Sebelum ngoding, tim Frontend WAJIB menambahkan aplikasi mereka ke dalam Project Firebase yang sudah dibuat oleh Backend. Buka link ini: <https://console.firebase.google.com/project/caritalent/settings/general>

### A. Untuk Tim Website

1. Di halaman Project Settings, scroll ke bawah ke bagian **"Your apps"**.
2. Klik tombol logo **Web (</>)**.
3. Masukkan nama aplikasi (misal: CariTalent Web App).
4. Klik **Register App**.
5. Google akan memunculkan sebuah *script* `firebaseConfig`. Simpan variabel `firebaseConfig` ini (berisi `apiKey`, `authDomain`, dll) karena akan dimasukkan ke dalam kodingan Website kamu nanti.

### B. Untuk Tim Mobile (Android)

1. Di halaman Project Settings, klik tombol logo **Android**.
2. Masukkan **Android package name** (contoh: `com.caritalent.app` — samakan dengan nama package di *project* Flutter/Android kamu).
3. Klik **Register App**.
4. Download file `google-services.json` yang diberikan.
5. Pindahkan file tersebut ke dalam folder aplikasi mobile kamu (khusus Flutter: taruh di `android/app/google-services.json`).

### C. Untuk Tim Mobile (iOS - Jika Ada)

1. Sama seperti Android, klik logo **iOS**.
2. Masukkan **Apple bundle ID**.
3. Download file `GoogleService-Info.plist`.
4. Masukkan ke dalam folder `ios/Runner` menggunakan XCode.

## 3. Dokumentasi Endpoint Backend

Frontend hanya perlu memanggil **SATU** endpoint ini setelah mendapatkan token dari Firebase (langkah kodingannya ada di bawah).

**Endpoint:** `PUT /api/v1/users/fcm-token`

**Auth:** Bearer Token (User harus sudah login)

**Header:** `Content-Type: application/json`

**Body Payload:**

```
{
  "fcm_token": "string_token_panjang_dari_firebase"
}
```

**Contoh Response Sukses (200 OK):**

```
{
  "success": true,
  "message": "FCM Token berhasil diperbarui",
  "data": {
    "fcm_token": "string_token_panjang_dari_firebase"
  }
}
```

## 4. Langkah Integrasi Koding untuk Frontend WEB (React/Vue/Vanilla)

Pastikan kamu sudah mendaftarkan app Web di konsol (Langkah 2A).

### 1. Install Firebase SDK:

```
npm install firebase
```

### 2. Inisialisasi Firebase & Minta Token:

```
import { initializeApp } from "firebase/app";
import { getMessaging, getToken, onMessage } from "firebase/messaging";

// COPY PASTE hasil dari Langkah 2A di sini
const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT.firebaseio.com",
  projectId: "YOUR_PROJECT_ID",
  messagingSenderId: "YOUR_SENDER_ID",
  appId: "YOUR_APP_ID"
};

const app = initializeApp(firebaseConfig);
const messaging = getMessaging(app);

// Minta permission dan token dari user (panggil fungsi ini setelah Login)
Notification.requestPermission().then((permission) => {
  if (permission === 'granted') {
    // Catatan: vapidKey bisa didapat di Project Settings -> Cloud Messaging -> Web configuration
    getToken(messaging, { vapidKey: 'YOUR_PUBLIC_VAPID_KEY' }).then((currentToken) => {
      if (currentToken) {
        console.log("Token FCM didapatkan:", currentToken);

        // TODO: Kirim currentToken ini ke API backend (PUT /api/v1/users/fcm-token)
      }
    });
  }
});
```

## 5. Langkah Integrasi Koding untuk Frontend MOBILE (Flutter)

Pastikan kamu sudah mendaftarkan app Android/iOS dan menaruh file json/plist di foldernya masing-masing (Langkah 2B & 2C).

### 1. Install Package:

```
flutter pub add firebase_core firebase_messaging
```

### 2. Inisialisasi & Ambil Token (di main.dart):

```

import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_messaging/firebase_messaging.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(); // Ini otomatis membaca file google-services.json

  FirebaseMessaging messaging = FirebaseMessaging.instance;

  // Minta izin notifikasi (khusus iOS, Android 13+ otomatis minta izin)
  NotificationSettings settings = await messaging.requestPermission();

  if (settings.authorizationStatus == AuthorizationStatus.authorized) {
    // Dapatkan Token
    String? token = await messaging.getToken();
    print("FCM Token: $token");

    // TODO: Kirim variabel 'token' ini ke API backend (PUT /api/v1/users/fcm-token)
  }

  // (Opsional) Listener jika notifikasi masuk SAAT aplikasi sedang asyik dibuka
  FirebaseMessaging.onMessage.listen((RemoteMessage message) {
    print('Notifikasi masuk: ${message.notification?.title}');
  });
}

```

## Catatan Penting

Setiap kali user melakukan **Logout**, tim Frontend sangat disarankan untuk menembak endpoint `PUT /api/v1/users/fcm-token` dengan payload `{"fcm_token": ""}` (dikosongkan). Tujuannya agar user yang sudah logout tidak tiba-tiba menerima notifikasi "nyasar" dari akun lamanya jika HP tersebut dipakai orang lain.